

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

BỘ MÔN NHẬP MÔN JAVASCRIPT



**BÀI TẬP LỚN NHẬP MÔN
JAVASCRIPT**

Giảng viên hướng dẫn	: Trần Minh Hiếu
Họ và tên sinh viên	: Nguyễn Đức Hiếu
	: Nguyễn Nam Khánh
	: Khuất Tiến Quang
Lớp tín chỉ	: RIPT1302-20242-01
Nhóm	: 1

Hà Nội – 2025

I. Thành viên nhóm

Tên thành viên	Chức vụ	Mã sinh viên	Nhiệm vụ
Nguyễn Đức Hiếu	Nhóm trưởng	B24DCCC118	HTML, CSS, JS, báo cáo
Nguyễn Nam Khánh	Thành viên	B24DCCC160	HTML, CSS, JS, NodeJS, báo cáo, powerpoint
Khuất Tiến Quang	Thành viên	B24DCCC233	HTML, CSS

II. Giới thiệu đề tài

1. Lý do chọn đề tài

- Cờ vua là trò chơi trí tuệ phổ biến rèn luyện tư duy logic.
- Phù hợp làm dự án web.
- Kết hợp kiến thức lập trình giao diện, xử lý nước đi, lưu lịch sử...

2. Mục tiêu của đề tài

- Xây dựng website cho phép người dùng chơi cờ vua trực tuyến.
- Giao diện dễ sử dụng, có đủ các tính năng của game cờ vua.
- Có các tính năng phụ như đếm thời gian, chơi với máy,...

3. Phạm vi thực hiện

- Giao diện cho người chơi.
- Logic xử lý luật cờ vua.
- Các tính năng bộ đếm giờ, phong quân, nhập thành,...
- Chơi với máy cơ bản.

4. Công nghệ sử dụng

- Frontend: HTML, CSS, JavaScript.
- Backend: Nodejs, Express, Socket.IO.

III. Phân tích hệ thống

1. Đối tượng sử dụng

- Bất kỳ ai sử dụng với mục đích giải trí, rèn luyện tư duy logic,...
- Sử dụng trên trình duyệt máy tính, thiết bị di động.

2. Các chức năng chính của hệ thống

- Hiện thị giao diện bàn cờ.
- Di chuyển quân đúng luật cờ vua.
- Phong cấp, nhập thành, lưu lịch sử nước đi.
- Chơi với máy cơ bản.
- Xác định kết quả thắng/thua.

IV. Thiết kế hệ thống

1. Giao diện người dùng (UI)

- Thanh Sidebar cố định dạng thu gọn/mở rộng.
- Bàn cờ vẽ bằng thẻ canvas.
- Khung bên phải gồm: bộ đếm thời gian, lịch sử nước đi, chế độ chơi, nút Play chính.
- Modal phong cấp quân.

2. Thiết kế điều hướng

- Mỗi mục trong Sidebar điều hướng tới trang riêng: puzzle, learn, social,...

3. Thiết kế chức năng chính

- Luật chơi được xử lý bằng file [chess.js](#)
- Hiển thị thời gian thực ván đấu.
- Lưu lịch sử nước đi.

V. Triển khai hệ thống

1. Xử lý giao diện (HTML/CSS)

1.1 HTML

```
<nav class="sidebar">
  <div class="logo">CHESSENA</div>
  <ul class="nav-links">
    <li><a href="chess.html"><span class="icon">🎮</span><span
class="text">Play</span></a></li>
    ...
  </ul>
</nav>
```

- <nav>: đại diện cho vùng điều hướng chính.
- <div class="logo">CHESSENA</div>: hiện tên/logo của trang web.
- <ul class="nav-links">: danh sách (không theo thứ tự) các mục điều hướng.
- Mỗi <a>... là một mục menu.

```
<div class="container">
  <div id="board-container">
    ...
  <div id="right-panel">
    ...
    <button id="main-play-button">Play</button>
    ...
  </div>
```

- Giao diện chính chia thành:

- <div id="board-container">: bàn cờ.
- <div id="right-panel">: bảng panel chứa các tiện ích bên phải.

`<button id="main-play-button">Play</button>`: nút Play chính.

```
<canvas id="board" width="540" height="540"></canvas>
```

- Vẽ bàn cờ bằng `<canvas>`.

```
<div id='clock-container'>
  <div id='black-clock'></div>
  <div id='white-clock'></div>
</div>
```

- Đồng hồ cho 2 bên: Đen và trắng.

```
<div id="turn" style="color:white; font-size:18px; margin-bottom:
10px;">Turn: White</div>
```

- Hiện thị lượt đi: Đen/trắng

```
<div id="move-history">
  <h3>LỊCH SỬ NƯỚC ĐI</h3>
  <ol id="moves-list"></ol>
</div>
```

- `<ol>`: Hiện thị nước đi dưới dạng danh sách có thứ tự.

- JavaScript sẽ cập nhật nội dung vào `<ol id="moves-list">`.

```
<label>Chọn chế độ:</label>
<select id="mode-dropdown">
  <option value="">-- Chọn chế độ --</option>
  <option value="pvp">PvP</option>
  <option value="ai">Chơi với Máy</option>
</select>
```

- Cho phép người chơi chọn chế độ chơi có sẵn.

- `<label>Chọn chế độ:</label>`: văn bản hiển thị cho người dùng biết đây là chỗ chọn chế độ chơi.

- `<select id="mode-dropdown">`: tạo hộp chọn (drop down) giúp người chơi chọn giữa các chế độ.

- Mỗi `<option>` là một lựa chọn chế độ chơi.

```
<button id="main-play-button">Play</button>
```

- Nút để bắt đầu game sau khi đã chọn chế độ chơi.

```
<div id="promotion-modal" class="promotion-modal">
  <p>Chọn quân để phong cấp</p>
  <img src="" alt="q" onclick="promote('q')" class="promote-piece">
  ...
</div>
```

- Bảng phong quân xuất hiện khi quân tốt đủ điều kiện phong cấp.
- Mỗi `<img src="">` là một hình ảnh đại diện cho quân cờ có thể phong cấp (đã dùng js để gán src).
- `onclick="promote('q')"` gọi hàm JavaScript `promote()` với tham số tương ứng khi người chơi bấm vào ảnh.
- `alt`: Hiện thị văn bản thay thế nếu ảnh không tải được (lỗi hoặc mất file).

```
<script src="chess.js"></script>
```

- Những file js tên [chess.js](#) chứa toàn bộ logic xử lý game.

```
<div id="introduction">
  <h1>Hướng Dẫn Chơi Cờ Vua</h1>
  <p>...</p>
  <ul>
    <li><strong>...</strong>...</li>
    ...
  </ul>
</div>
```

- `<div id="introduction">`: là thẻ chứa toàn bộ nội dung phần giới thiệu.
- `<h1>`: là tiêu đề lớn nhất, đại diện cho tiêu đề nội dung chính.
- `<p>`: thẻ đoạn văn chứa nội dung.
- `<ul>` và `<li>`: là danh sách chứa từng mục cụ thể.
- `<strong>`: để in đậm, nhấn mạnh nội dung.

```
<div id="piece-rules">
  <h2>...</h2>
  <div class="introduction">
    <h3>...</h3>
    <p><strong>
      ...
    </p>
    <img src...>
  </div>
```

- Mỗi mục `<div>` chứa cách di chuyển, bắt quân của từng quân cờ và kèm theo phần hình ảnh `<img src>`.
- Tương tự cho mỗi quân cờ khác.

```
</div>
<a href="chess.html" id="button">Chơi</a>
```

</div>

- Cuối cùng sau khi đọc qua toàn bộ luật chơi, cuối cùng là 1 button đưa về giao diện chơi chính.

1.2 CSS

Để tạo giao diện trực quan và hiện đại cho trang web chơi cờ vua, nhóm đã sử dụng ngôn ngữ CSS để định dạng bố cục và hiệu ứng cho toàn bộ website. Các điểm chính trong thiết kế bao gồm:

- **Nền và bố cục tổng thể:** Giao diện có nền gradient kết hợp ảnh nền bàn cờ để tạo cảm giác sinh động, chuyên nghiệp. Toàn bộ trang được căn chỉnh bằng **flexbox**, giúp các thành phần như bàn cờ, lịch sử nước đi, đồng hồ,... hiển thị cân đối, dễ quan sát trên nhiều độ phân giải.
- **Thanh điều hướng (sidebar):** Được đặt cố định bên trái, mặc định thu gọn chỉ hiển thị biểu tượng, khi rê chuột vào sẽ mở rộng hiển thị văn bản. Hiệu ứng chuyển động (**transition**) tạo cảm giác mượt mà. Mỗi mục trong menu có biểu tượng riêng như 🎮 (Play), 🧩 (Puzzles), 📖 (Learn)... giúp người dùng dễ nhận diện.
- **Bàn cờ:** Được vẽ bằng thẻ `<canvas>` đặt chính giữa, có đổ bóng (**box-shadow**) và bo góc (**border-radius**) tạo chiều sâu.
- **Đồng hồ và lịch sử nước đi:** Thiết kế với nền tối, chữ sáng, sử dụng font **monospace** giúp dễ theo dõi. Lịch sử có thanh cuộn tùy chỉnh bằng CSS, hiển thị đẹp trên cả trình duyệt hỗ trợ **scrollbar**.
- **Modal phong cấp:** Khi tốt phong cấp, một hộp thoại hiện ra giữa màn hình với 4 hình ảnh quân cờ có thể chọn. Giao diện này sử dụng **position: fixed** và **transform** để căn giữa chính xác.
- **Nút bắt đầu và chế độ chơi:** Giao diện có nút "Play" thiết kế với ảnh nền riêng và hiệu ứng khi rê chuột. Người chơi có thể chọn chế độ PvP hoặc chơi với máy thông qua **select box** được định dạng thẩm mỹ.
- **Giao diện học cờ (learn.html):** Các phần nội dung học luật cờ được hiển thị trong các khối trắng (**whitesmoke**) có viền và đổ bóng. Hình ảnh quân cờ có kích thước phù hợp, căn giữa và viền rõ ràng.

2. Xử lý luật chơi (JavaScript)

2.1 Khởi tạo các biến

```
const canvas = document.getElementById("board");  
const ctx = canvas.getContext("2d");
```

- Truy cập thẻ `<canvas id="board">` trong HTML và khởi tạo `ctx` để thao tác vẽ 2D.

```

const SQUARE_SIZE = 67.5; // Kích thước mỗi ô cờ (px)
const DIMENSION = 8; // Bàn cờ 8x8 ô
const LEFT_MARGIN = 20; // Lệ trái của bàn cờ
const RIGHT_MARGIN = 20; // Lệ phải
const BOARD_TOP = 120; // Lệ trên (dùng cho đồng hồ, nút, v.v.)
const BOARD_BOTTOM = 120; // Lệ dưới

const BEIGE = "#f0d9b5"; // Màu ô sáng
const BROWN = "#b58863"; // Màu ô tối

```

- Khai báo các hằng số điều chỉnh linh hoạt bố cục và phối màu bàn cờ

```

const canvasWidth = SQUARE_SIZE * DIMENSION + LEFT_MARGIN + RIGHT_MARGIN;
const canvasHeight = SQUARE_SIZE * DIMENSION + BOARD_TOP + BOARD_BOTTOM;
canvas.width = canvasWidth;
canvas.height = canvasHeight;

const boardOffsetX = LEFT_MARGIN;
const boardOffsetY = BOARD_TOP;

```

- Tính chiều dài, rộng của canvas và đặt lại width, height đúng với kích thước tính toán.
 - Tọa độ ô đầu tiên (A8) để vẽ bàn cờ, các ô sau dịch theo SQUARE_SIZE.

```

let board = [
  ["br", "bn", "bb", "bq", "bk", "bb", "bn", "br"],
  ["bp", "bp", "bp", "bp", "bp", "bp", "bp", "bp"],
  ["_", "_", "_", "_", "_", "_", "_", "_"],
  ["_", "_", "_", "_", "_", "_", "_", "_"],
  ["_", "_", "_", "_", "_", "_", "_", "_"],
  ["_", "_", "_", "_", "_", "_", "_", "_"],
  ["wp", "wp", "wp", "wp", "wp", "wp", "wp", "wp"],
  ["wr", "wn", "wb", "wq", "wk", "wb", "wn", "wr"]
];

```

- Khởi tạo trạng thái cơ bản của bàn cờ, vị trí đứng của các quân cờ ban đầu.

```

let selected = null;
let validMoves = [];
let moveHistory = [];
let whiteLost = [];
let blackLost = [];

let turn = "w";
let gameOver = false;

let gameMode = null;
let gameStarted = false;

let promotionRow = null;
let promotionCol = null;
let promotionColor = null;

```

- Khởi tạo các biến trạng thái quản lý trò chơi, lượt đi, chế độ chơi,...

```
let whiteTime = 600;
let blackTime = 600;
let timerInterval = null;
```

- Khởi tạo đồng hồ thi đấu, thời gian mặc định 10'
- Lưu ID của bộ đếm `setInterval`.

```
let moved = {
  wk: false, bk: false,
  wr1: false, wr2: false,
  br1: false, br2: false
};
```

- Kiểm tra vua và xe đã di chuyển chưa, xử lý hợp lệ của nước nhập thành.

```
const moveSound = new Audio('sounds/move.mp3');
const captureSound = new Audio('sounds/capture.mp3');
```

- Khởi tạo âm thanh cho nước di chuyển và bắt quân.

2.2 Hàm khởi tạo ảnh quân cờ (*loadImages*)

```
let pieceImages = {};

function loadImages() {
  const types = ["wp", "wr", ..., "bk"];
  let count = 0;

  types.forEach(type => {
    const img = new Image();
    img.src = `images/${type}.png`;
    img.onload = () => {
      pieceImages[type] = img;
      if (++count === types.length) drawBoard();
    };
  });
}
```

- Tạo object lưu trữ hình ảnh quân cờ.
- `types`: mảng chứa mã của tất cả 12 loại quân cờ.
- Duyệt qua từng phần tử trong mảng bằng `types.forEach` rồi tạo một đối tượng ảnh `<img>` mới trong bộ nhớ.
- Gán đường dẫn ảnh `img.src` theo tên quân cờ và hàm xử lý sự kiện `img.onload`.
- Lưu đối tượng ảnh đã tạo được vào `pieceImages`, sau khi đủ 12 ảnh gọi hàm `drawBoard()` để vẽ bàn cờ hoàn chỉnh.

2.3 Hàm vẽ bàn cờ (*drawBoard*)

```
ctx.fillStyle = 'rgba(145, 152, 203, 0.54)';
ctx.fillRect(0, 0, canvas.width, canvas.height);
```

- Vẽ nền canvas độ trong suốt 54%.


```
for (let row = 0; row < DIMENSION; row++) {
  for (let col = 0; col < DIMENSION; col++) {
    const x = boardOffsetX + col * SQUARE_SIZE;
    const y = boardOffsetY + row * SQUARE_SIZE;
```

- Vòng lặp lồng duyệt qua từng ô trên bàn cờ và xác định tọa độ (x, y) để vẽ 1 ô.

```
const color = (row + col) % 2 === 0 ? BEIGE : BROWN;
ctx.fillStyle = color;
ctx.fillRect(x, y, SQUARE_SIZE, SQUARE_SIZE);
```

- Mỗi ô cờ được xen kẽ giữa màu sáng BEIGE và tối BROWN.

```
if (selected && validMoves.some(m => m[0] === r && m[1] === c)) {
  ctx.fillStyle = "rgba(0,255,0,0.3)";
  ctx.fillRect(x, y, SQUARE_SIZE, SQUARE_SIZE);
}
```

- Nếu người dùng đã chọn quân và tọa độ ô đó là nước đi hợp lệ, tô ô đó màu xanh.

```
const piece = board[r][c];
if (piece !== "__") {
  ctx.drawImage(pieceImages[piece], x, y, SQUARE_SIZE, SQUARE_SIZE);
}
```

- Nếu ô chứa quân cờ không trống, vẽ ảnh quân cờ tại vị trí đó với kích thước phù hợp.

```
for (let c = 0; c < DIMENSION; c++) {
  const x = boardOffsetX + c * SQUARE_SIZE + SQUARE_SIZE / 2;
  const y = boardOffsetY + DIMENSION * SQUARE_SIZE + 10;
  ctx.fillText(files[c], x, y);
}

for (let r = 0; r < DIMENSION; r++) {
  const y = boardOffsetY + r * SQUARE_SIZE + SQUARE_SIZE / 2;
  const x = LEFT_MARGIN / 2;
  ctx.fillText(8 - r, x, y);
}
```

- Vẽ tên cột (a-h) bên dưới bàn cờ và vẽ số hàng (1-8) ở lề trái bàn cờ.

```
drawPlayerInfo();
```

- Ở cuối hàm, gọi hàm drawPlayerInfo() để cập nhật toàn bộ giao diện sau mỗi thay đổi.

2.4 Hàm vẽ thông tin người chơi và quân bị bắt (drawPlayerInfo)

```
function drawPlayerInfo() {
  const avatarImg2 = new Image();
  avatarImg2.src = 'images/player2.png';
  avatarImg2.onload = () => ctx.drawImage(avatarImg2, 10, blackY -
```

```

avatarSize / 2, avatarSize, avatarSize);

ctx.fillText("Người chơi 2 (Đen)", 70, blackY);

whiteLost.forEach(piece => {
  if (pieceImages[piece]) {
    ctx.drawImage(pieceImages[piece], bx, blackY + avatarSize / 2 - 10,
25, 25);
    bx += 28;
  }
});

```

- Tạo đối tượng ảnh và gán đường dẫn ảnh cho người chơi 2 (black).
- Sau khi tải ảnh xong, vẽ avatar `drawImage` và nhập tên `ctx.fillText` tại vị trí theo kích thước mong muốn.
- Duyệt qua từng quân bắt được từ người chơi 1 rồi vẽ tại vị trí theo kích thước mong muốn.
- Tương tự với người chơi 1 (white).

2.5 Hàm cập nhật hiển thị (*updateSidebar*)

```

function updateSidebar() {
  document.getElementById("turn").textContent = `Turn: ${turn === "w" ?
"White" : "Black"}`;
  document.getElementById("white-lost").textContent = whiteLost.join(" ");
  document.getElementById("black-lost").textContent = blackLost.join(" ");
}

```

- Hàm gán nội dung lượt đi và quân cờ bị ăn.

2.6 Hàm cập nhật lịch sử nước đi (*updateMoveHistory*)

```

function updateMoveHistory() {
  const movesList = document.getElementById('moves-list');
  movesList.innerHTML = '';

  for (let i = 0; i < moveHistory.length; i += 2) {
    const whiteMove = moveHistory[i];
    const blackMove = moveHistory[i + 1];
    const li = document.createElement('li');
    li.innerHTML = `
      ${whiteMove ? getMoveIcon(whiteMovenotation, whiteMove.color) :
''}
      ${blackMove ? ' | ' + getMoveIcon(blackMovenotation,
blackMove.color) : ''}
    `;
    movesList.appendChild(li);
  }
  movesList.scrollTop = movesList.scrollHeight;
}

```

- Lấy thẻ `<ol>` chứa danh sách đi và xóa toàn bộ nội dung cũ trước khi cập nhật.
- Duyệt qua lịch sử các nước đi trong ván cờ (mảng `moveHistory`) và chia ra thành từng lượt đầy đủ gồm trắng, đen.

- Tạo thêm 1 danh sách mới để chứa 1 cặp nước đi. Hiển thị lịch sử nước đi và icon, thêm vào danh sách.
- `movesList.scrollTop = movesList.scrollHeight`: tự động cuộn xuống cuối cùng.

2.7 Hàm tạo Icon cho quân cờ (*getMoveIcon*)

```
function getMoveIcon(move, color) {
  const pieceMap = {R: 'r', N: 'n', B: 'b', Q: 'q', K: 'k'};

  const pieceChar = move[0];
  const square = pieceChar.match(/[RNBQK]/) ? move.slice(1) : move;

  const piece = pieceMap[pieceChar] || 'p';
  const imgSrc = `images/${color}${piece}.png`;

  return `
    
    ${square}
  `;
}
```

- Tạo ký hiệu, lấy ký tự đầu tiên của mỗi chuỗi move.
- Kiểm tra `pieceChar` có phải các quân cờ (R, N, B, Q, K), nếu đúng lấy từ ký tự thứ 2, nếu không lấy toàn bộ rồi gắn với mã quân cờ tương ứng.
- Tạo đường dẫn ảnh dựa trên màu và loại quân, trả về chuỗi HTML gồm icon quân cờ và lịch sử nước đi.

2.8 Hàm xác định tọa độ chuột (*getSquare*)

```
function getSquare(x, y) {
  return [
    Math.floor((y - boardOffsetY) / SQUARE_SIZE),
    Math.floor((x - boardOffsetX) / SQUARE_SIZE)
  ];
}
```

- Lấy tọa độ pixel (x, y) trừ đi cho khoảng lề (`boardOffsetX`, `boardOffsetY`) chia cho thích thước ô vuông (`SQUARE_SIZE`) rồi dùng `Math.floor()` chia làm tròn xuống, ta được tọa độ con trỏ chuột click.

2.9 Hàm trả về danh sách các nước đi hợp lệ của quân cờ (*getMoves*)

```
const type = piece[1]; // 'p', 'r', 'n', ...
const color = piece[0]; // 'w' hoặc 'b'
const moves = [];
```

- Tách loại màu [1], quân cờ [2] và khởi tạo danh sách nước đi hợp lệ.

```
const isEnemy = (row, col) => board[row][col] !== "___" &&
board[row][col][0] !== color;
```

```
const isEmpty = (row, col) => board[row][col] === "___";
```

- Hàm trả về giá trị `isEnemy` nếu tại ô đó có quân cờ khác màu, trả về giá trị `isEmpty` nếu ô trống.

```
const add = (dr, dc, repeat = false) => {
  let row = r + dr, col = c + dc;
  while (row >= 0 && row < 8 && col >= 0 && col < 8) {
    if (isEmpty(row, col)) {
      moves.push([row, col]);
    } else {
      if (isEnemy(row, col)) moves.push([row, col]);
      break;
    }
    if (!repeat) break;
    row += dr;
    col += dc;
  }
};
```

- Hàm `add(dr, dc, repeat = false)` là tiện ích giúp thêm nước đi cho quân theo hướng. `dr, dc` là độ dịch theo dòng cột tùy theo cách di chuyển của từng quân cờ. `repeat` nếu được đặt thành `true` sẽ cho phép quân di chuyển được nhiều ô liên tiếp.

- Phạm vi di chuyển được tính bắt đầu từ tọa độ quân cờ, trong phạm vi bàn cờ, nếu gặp phải ô trống hoặc quân địch thì thêm tọa độ vào `moves`, còn lại thì dừng vòng lặp.

```
case "p": {
  const dir = color === "w" ? -1 : 1;
  const start = color === "w" ? 6 : 1;
```

- Với trường hợp tốt, vị trí tốt trắng bắt đầu từ hàng 6, đi lên (-1), tốt đen bắt đầu từ hàng 1, đi xuống (+1).

```
if (isEmpty(r + dir, c)) {
  moves.push([r + dir, c]);

  if (r === startRow && isEmpty(r + 2 * dir, c)) {
    moves.push([r + 2 * dir, c]);
  }
}
```

- Nếu trước mặt tốt đang đứng là ô trống, tốt được đi thẳng 1 ô.

- Nếu vị trí ban đầu của tốt là vị trí bắt đầu, tốt được đi thẳng 2 ô.

```
[-1, 1].forEach(dc => {
  const nr = r + dir, nc = c + dc;
  if (nr >= 0 && nr < 8 && nc >= 0 && nc < 8 && isEnemy(nr, nc)) {
    moves.push([nr, nc]);
  }
});
```

- Kiểm tra ô chéo trước mặt tốt, nếu là quân địch nằm trong bàn cờ, tốt được phép ăn.

```
case "r":
  [[1, 0], [-1, 0], [0, 1], [0, -1]].forEach(([dr, dc]) => add(dr, dc, true));
  break;

case "b":
  [[1, 1], [-1, -1], [1, -1], [-1, 1]].forEach(([dr, dc]) => add(dr, dc, true));
  break;

case "q":
  [[1, 0], [-1, 0], [0, 1], [0, -1], [1, 1], [-1, -1], [1, -1], [-1, 1]].forEach(([dr, dc]) => add(dr, dc, true));
  break;
```

- Với các quân xe, tượng, hậu, độ dịch sẽ được đặt tương ứng với các tọa độ di chuyển ngang, dọc và chéo. repeat được xét thành true để có thể tạo thành vòng lặp cho các nước di chuyển liên tiếp.

```
case "k":
  [[1, 0], [-1, 0], [0, 1], [0, -1], [1, 1], [-1, -1], [1, -1], [-1, 1]].forEach(([dr, dc]) => add(dr, dc));
  if (canCastle(r, "kingside")) moves.push([r, 6]);
  if (canCastle(r, "queenside")) moves.push([r, 2]);
  break;
```

- Trường hợp với quân vua sẽ di chuyển 8 ô xung quanh, bên cạnh đó là nước nhập thành cánh hậu và cánh vua với điều kiện thỏa mãn hàm canCastle() về vị trí theo luật.

```
case "n":
  [[2, 1], [2, -1], [-2, 1], [-2, -1], [1, 2], [1, -2], [-1, 2], [-1, -2]].forEach(([dr, dc]) => {
    const nr = r + dr;
    const nc = c + dc;
    if (nr >= 0 && nr < 8 && nc >= 0 && nc < 8 && (isEmpty(nr, nc) || isEnemy(nr, nc))) {
      moves.push([nr, nc]);
    }
  });
  break;
```

- Quân mã di chuyển theo hình chữ “L”, ko có vòng lặp.

2.10 Hàm kiểm tra điều kiện nhập thành (canCastle)

```
function canCastle(row, side) {
  if (turn === 'w' && row === 7) {
    if (side === 'kingside') {
      return !moved.wk && !moved.wr2 &&
        board[7][5] === "_" && board[7][6] === "_";
    } else {
      return !moved.wk && !moved.wr1 &&
        board[7][1] === "_" && board[7][2] === "_" && board[7][3] === "_";
    }
  }
  return false;
}
```

```
"_";  
}  
}
```

- Kiểm tra các điều kiện như đến lượt, xe và vua chưa di chuyển, giữa xe và vua không có quân cờ nào,...

2.11 Hàm kiểm tra điều kiện phong quân (*checkPromotion*)

```
function checkPromotion(row, col, piece) {  
  if ((piece === 'wp' && row === 0) || (piece === 'bp' && row === 7)) {  
    promotionRow = row;  
    promotionCol = col;  
    promotionColor = piece[0];  
    document.getElementById('promotion-modal').style.display = 'block';  
    document.querySelectorAll('#promotion-modal img').forEach(img => {  
      const type = img.alt.toLowerCase().charAt(0);  
      img.src = `images/${promotionColor}${type}.png`;  
    });  
  }  
}
```

- Kiểm tra vị trí thỏa mãn điều kiện phong quân của tốt đen/trắng rồi ghi nhớ màu và vị trí quân tốt.
- Hiện thị phần tử HTML có id="promotion-modal" (modal popup để chọn quân mới).
- Cập nhật ảnh phù hợp với màu quân (promotionColor) và loại quân (type) .

2.12 Hàm phong quân (*promote*)

```
function promote(newPiece) {  
  board[promotionRow][promotionCol] = promotionColor + newPiece;  
  document.getElementById('promotion-modal').style.display = 'none';  
  promotionRow = null;  
  promotionCol = null;  
  promotionColor = null;  
  drawBoard();  
}
```

- Thay thế quân tốt hiện tại bằng quân mới theo mã quân đã chọn.
- Ấn hộp thoại sau khi phong quân.
- Reset các giá trị phong quân để chuẩn bị cho lần tiếp theo.

2.13 Hàm chuyển tọa độ di chuyển thành ký hiệu nước đi (*toChessNotation*)

```
function toChessNotation(fromRow, fromCol, toRow, toCol, piece, captured)  
{  
  const files = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h'];  
  const ranks = ['8', '7', '6', '5', '4', '3', '2', '1'];  
  const to = files[toCol] + ranks[toRow];  
  const pieceChar = piece[1].toLowerCase() === 'p' ? '' :  
    piece[1].toUpperCase();  
  return pieceChar + to;  
}
```

- Hàm nhập vào các tham số tọa độ bắt đầu, tọa độ đích, mã quân và khởi tạo các tọa độ.
- Tạo ký hiệu ô đích đến và lấy ký tự đại diện cho quân di chuyển.
- Trả về ký hiệu nước đi.

2.14 Hàm di chuyển quân cờ (movePiece)

```
const piece = board[fromRow][fromCol];
const captured = board[toRow][toCol];

const isValid = getMoves(piece, fromRow, fromCol).some(m => m[0] === toRow
&& m[1] === toCol);
if (!isValid) return;
```

- Khởi tạo biến quân cờ và đích đến.
- Kiểm tra tính hợp lệ của nước đi quân được chọn hiện tại và tính hợp lệ của nước đi đích đến.

```
if (captured !== "__") {
  captureSound.play();
  if (captured[0] === "w") whiteLost.push(captured);
  else blackLost.push(captured);
} else {
  moveSound.play();
}
```

- Kiểm tra ô đích (toRow, toCol) có đang chứa quân cờ hay không.
- Nếu chứa quân cờ thì sẽ phát âm thanh captureSound và thêm quân đó vào quân bị ăn mất. Nếu không chứa quân cờ thì sẽ phát âm thanh moveSound.

```
moveHistory.push({
  notation: toChessNotation(fromRow, fromCol, toRow, toCol, piece,
captured),
  color: piece[0]
});
updateMoveHistory()
```

- moveHistory được cập nhật 2 thông tin gồm toChessNotation() thông tin của nước đi di chuyển đó và color màu sắc của quân cờ.

```
if (piece === "wk") {
  moved.wk = true;

  if (toRow === 7 && toCol === 6) {
    board[7][5] = board[7][7];
    board[7][7] = "__";
    moved.wr2 = true;
  } else if (toRow === 7 && toCol === 2) {
    board[7][3] = board[7][0];
    board[7][0] = "__";
    moved.wr1 = true;
  }
}
```

```
}
```

- Thực hiện bước còn lại của nhập thành, hoán đổi lại vị trí quân xe vào vị trí quân vua khi thực hiện nhập thành.
- Logic tương tự được xử lý với vua đen.
- Sau khi thực hiện, đánh dấu lại quân xe, và quân vua thực hiện nhập thành đã di chuyển.

```
checkPromotion(toRow, toCol, piece);  
turn = turn === "w" ? "b" : "w";
```

- Sau mỗi nước đi, kiểm tra xem có đến hàng phong cấp hay không để hiện hộp phong cấp, sau đó đổi lại lượt cho bên còn lại.

```
startTimer();  
updateSidebar();  
drawBoard();  
checkGameOver();
```

- Sau khi một nước đi đã được thực hiện, cập nhật lại toàn bộ trạng thái giao diện trò chơi.
- `startTimer()`: khởi động lại bộ đếm thời gian cho người chơi hiện tại.
- `updateSidebar()`: cập nhật lại phần giao diện.
- `drawBoard()`: Vẽ lại ô cờ, quân cờ, highlight nước đi hợp lệ.
- `checkGameOver()`: kiểm tra ván cờ kết thúc.

```
if (gameMode === 'ai' && turn === 'b' && !gameOver) {  
  setTimeout(() => {  
    makeAIMove();  
  }, 500);  
}
```

- Kích hoạt AI trong chế độ chơi với máy, lượt của đen và ván cờ chưa kết thúc.
- `setTimeout()` giúp delay thời gian, tạo cảm giác suy nghĩ của máy.

2.15 Hàm đồng hồ (*startTimer*)

```
function startTimer() {  
  if (timerInterval) clearInterval(timerInterval);  
  timerInterval = setInterval(() => {  
    if (gameOver) {  
      clearInterval(timerInterval);  
      return;  
    }  
  })  
}
```

- Khởi động bộ đếm giờ cho lượt chơi hiện tại, nếu đã có `timerInterval` đang chạy thì reset.
- Khởi tạo hàm lặp (1s), gán kết quả trả về cho `timerInterval`, nếu game kết thúc, dừng bộ đếm và thoát hàm.


```

if (turn === 'w') {
  whiteTime--;
  if (whiteTime <= 0) {
    clearInterval(timerInterval);
    showGameOver("Đen thắng! Trắng hết giờ.");
    gameOver = true;
    return;
  }
}

```

- Nếu đến lượt trắng, giảm biến `whitetime` 1 giây. Nếu hết giờ, tự động thông báo bên đen thắng.
- Logic tương tự với bên đen.

2.16 Hàm cập nhật đồng hồ và format (*updateClocks, formatTime*)

```

document.getElementById("white-clock").innerText = `🕒 Trắng:
${formatTime(whiteTime)}`;
document.getElementById("black-clock").innerText = `🕒 Đen:
${formatTime(blackTime)}`;
}

const m = String(Math.floor(seconds / 60)).padStart(2, '0');
const s = String(seconds % 60).padStart(2, '0');
return `${m}:${s}`;
}

```

- Cập nhật thời gian cho 2 bên trong giao diện và icon.
- Chuyển số giây thành dạng “pp:ss”.

2.17 Các hàm quản lý trạng thái trò chơi (*startGame, restartGame, quitGame, showGameOver*)

- Khởi tạo game theo chế độ, restart trò chơi, thoát trò chơi và hiển thị kết thúc.

2.18 Xử lý sự kiện khi click

```

canvas.addEventListener("click", e => {
  ...
});

```

- Đoạn mã xử lý các sự kiện trong giao diện sau khi click chuột gồm các hàm bên trên và hiển thị thông báo `showGameOver(message)` khi ván cờ kết thúc.

2.19 Xử lý sự kiện khi click Play

```

document.getElementById("main-play-button").addEventListener("click", ()
=> {
  const mode = document.getElementById("mode-dropdown").value;
  if (!mode) {
    alert("Vui lòng chọn chế độ chơi trước khi bắt đầu!");
    return;
  }
  startGame(mode);
}

```

```
});
```

- Gắn trình sự kiện click cho nút Play.
- Lấy giá trị được chọn `id="mode-dropdown"` với chế độ đã chọn để bắt đầu game.
- Nếu mode chưa có giá trị, hiện thông báo và ngừng thực hiện tiếp.

3. Tạo chế độ Online:

Trang web được xây dựng nhằm hỗ trợ hai người chơi có thể đánh cờ vua với nhau qua internet theo thời gian thực (real-time). Toàn bộ giao diện cờ vua được vẽ bằng **HTML5 Canvas**, còn phần xử lý truyền dữ liệu giữa hai người chơi được thực hiện bằng **Socket.IO** kết hợp với [Node.js](#).

A/Server.js

I. Cấu trúc hệ thống

Thành phần	Mô tả
<code>server.js</code>	Tạo server và xử lý truyền thông giữa 2 client (Socket.IO)
<code>online.js</code>	Xử lý giao diện người chơi, vẽ bàn cờ, gửi và nhận dữ liệu Socket.IO
<code>online.html</code>	Giao diện chính để người chơi vào phòng và bắt đầu trận đấu
Canvas + JavaScript	Vẽ bàn cờ, xử lý logic di chuyển, ăn quân, phong cấp,...

II. Chi tiết `server.js` – Node.js + Socket.IO

1. Khởi tạo server

```
const express = require("express");
const http = require("http");
const { Server } = require("socket.io");

const app = express();
const server = http.createServer(app);
const io = new Server(server);
```

2. Quản lý người chơi và phòng

- Mỗi người chơi khi truy cập sẽ chọn hoặc tạo một phòng (roomId)
- Dữ liệu ván cờ (board, move history,...) được lưu vào gameStates[roomId]

```
const gameStates = {};
```

```
io.on("connection", (socket) => {  
  socket.on("join", (roomId) => {  
    socket.join(roomId);  
    socket.data.roomId = roomId;  
  
    // Phân vai trắng hoặc đen  
    const players = io.sockets.adapter.rooms.get(roomId);  
    const color = players.size === 1 ? "w" : "b";  
    socket.emit("playerColor", color);  
  
    // Nếu đủ 2 người thì gửi gameState hiện tại  
    if (players.size === 2 && gameStates[roomId]) {  
      socket.emit("opponentMove", gameStates[roomId]);  
    }  
  });  
});
```

3. Xử lý gửi nhận nước đi

```
socket.on("move", (data) => {  
  const roomId = socket.data.roomId;  
  if (!roomId) return;  
  
  gameStates[roomId] = data;  
  socket.to(roomId).emit("opponentMove", data); // Gửi nước đi  
  cho đối thủ  
});
```

III. Chi tiết online.js – JavaScript giao diện

1. Kết nối đến server

```
const socket = io();  
socket.emit("join", roomId);
```

2. Nhận vai trò người chơi

```
socket.on("playerColor", (color) => {  
  myColor = color; // "w" hoặc "b"  
});
```

3. Xử lý khi người chơi đi nước cờ

```
function sendMove(from, to) {  
  const moveData = {  
    from,  
    to,  
    board: currentBoard,  
    whiteLost,  
    blackLost,  
    moved,  
    moveHistory,  
    turn: nextTurn  
  };  
  
  socket.emit("move", moveData);  
}
```

4. Xử lý khi nhận nước đi từ đối phương

```
socket.on("opponentMove", (data) => {  
  console.log("👉 opponentMove received", data);  
  updateBoard(data.board);  
  moveHistory = data.moveHistory;  
  drawBoard(); // Vẽ lại bàn cờ  
});
```

IV. Luồng hoạt động tổng quát

1. Người chơi A và B vào trang web → Nhận cùng Room ID → Tham gia cùng phòng
2. Server xác định người vào trước là Trắng (w), người sau là Đen (b)
3. Khi Trắng đi → gửi dữ liệu lên server → server truyền sang cho Đen
4. Bên Đen nhận dữ liệu và vẽ lại bàn cờ (ngược lại tương tự)
5. Game kết thúc khi chiếu hết hoặc đầu hàng (người chơi bên kia out ra khỏi web)

B/Online.js

1. Kết nối Socket.IO

```
const socket = io();
```

Chức năng: Khởi tạo kết nối WebSocket giữa trình duyệt của người chơi và server.

Vai trò: Giúp trao đổi dữ liệu giữa hai người chơi qua server trung gian sử dụng Socket.IO.

2. Gửi yêu cầu tham gia phòng chơi

```
socket.emit("join", roomId);
```

Chức năng: Gửi sự kiện **"join"** lên server kèm theo **roomId** mà người chơi nhập.

Vai trò: Ghép hai người chơi vào cùng một phòng chơi online, giúp server phân biệt các ván cờ khác nhau.

3. Nhận màu quân cờ từ server

```
socket.on("playerColor", (color) => {  
  myColor = color;  
  console.log("Bạn là bên", myColor === "w" ? "Trắng" : "Đen");  
});
```

Chức năng: Nhận sự kiện "playerColor" từ server để biết mình là bên trắng ("w") hay bên đen ("b").

Vai trò: Xác định vai trò của người chơi trong ván đấu để xử lý lượt đi hợp lệ, vẽ hướng quân, chọn nước đi hợp lệ,...

4. Gửi nước đi đến server

```
socket.emit("move", {  
  from: selectedFrom,  
  to: [row, col],  
  board: drawBoard(),      // Bàn cờ sau khi di chuyển  
  whiteLost: [...whiteLost], // Quân trắng bị mất  
  blackLost: [...blackLost], // Quân đen bị mất  
  moved: { ...moved },      // Trạng thái đã di chuyển (phục vụ castling)  
  moveHistory: [...moveHistory], // Lịch sử nước đi  
  turn          // Lượt tiếp theo  
});
```

Chức năng: Gửi toàn bộ trạng thái của ván cờ sau mỗi nước đi đến server.

Vai trò: Giúp đồng bộ hóa trạng thái bàn cờ giữa hai người chơi trong cùng phòng chơi.

5. Nhận nước đi từ đối phương

```
socket.on("opponentMove", (data) => {  
  console.log("📥 opponentMove received", data);  
  
  if (!imagesLoaded) {  
    pendingMove = data;  
    console.log("⌚ Images not loaded, saving to pendingMove");  
  }  
});
```

```
    return;
}

// Cập nhật lại toàn bộ trạng thái bàn cờ
currentBoard = data.board;
whiteLost = data.whiteLost;
blackLost = data.blackLost;
moved = data.moved;
moveHistory = data.moveHistory;
turn = data.turn;

drawBoard(); // Vẽ lại bàn cờ dựa trên dữ liệu mới nhận được
});
```

Chức năng: Nhận sự kiện "opponentMove" từ server và cập nhật trạng thái bàn cờ.

Vai trò:

Cập nhật bàn cờ theo đúng nước đi của đối thủ.

Đảm bảo đồng bộ thời gian thực giữa hai người chơi.

Giúp ván cờ tiếp tục đúng trạng thái mà đối thủ vừa thực hiện.

6. Xử lý tình huống ảnh chưa tải kịp

```
if (!imagesLoaded) {
    pendingMove = data;
    return;
}
```

Chức năng: Trì hoãn áp dụng nước đi từ đối thủ nếu hình ảnh quân cờ chưa sẵn sàng.

Vai trò: Tránh lỗi khi cố vẽ bàn cờ mà hình ảnh chưa được tải, đảm bảo trải nghiệm mượt mà.

C/Online.HTML

```
<script src="/socket.io/socket.io.js"></script>
<script src="online.js"></script>
```

Tác dụng:

Giúp trình duyệt (client) có thể kết nối tới server qua WebSocket hoặc fallback như polling.

File JavaScript, nơi viết logic xử lý bàn cờ, nước đi, vẽ lại canvas, và quan trọng nhất: kết nối và xử lý socket từ client.

D/ Quy trình để cho project lên được web để chơi online

Tạo package . json (chạy lệnh này trong Terminal)

```
npm init -y
```

Thêm express

```
npm install express
```

Push lên GITHUB

Mở Terminal tại thư mục chứa project:

```
git init
git add .
git commit -m "Initial commit"
```

Tạo một repo trên GitHub (<https://github.com/NamKhanh1811/Chess-web>)

Kết nối và push:


```
git remote add origin https://github.com/your-username/chess-online.git
git branch -M main
git push -u origin main
```

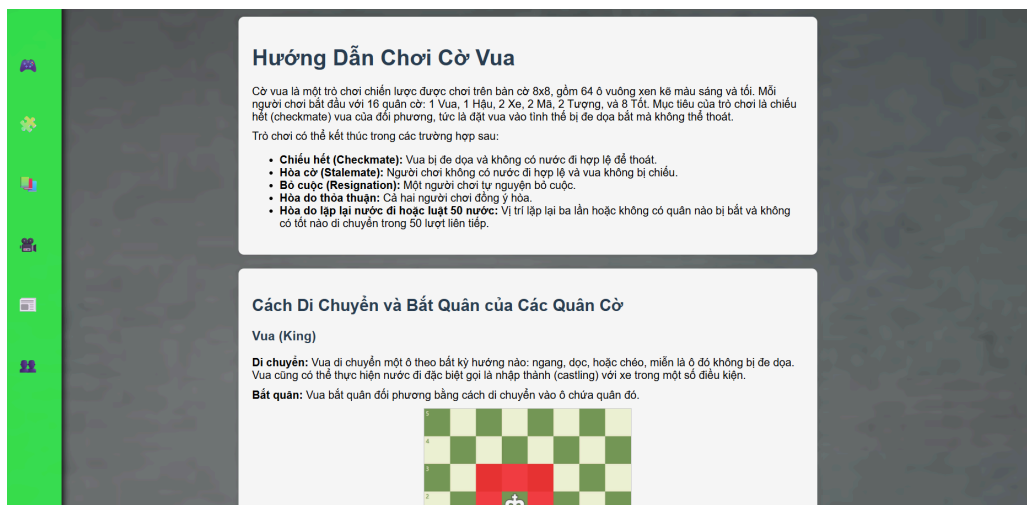
Cuối cùng là deploy trên Render để web chạy được chơi online vì GITHUB ko hỗ trợ BACKEND

4. Các tính năng khác:

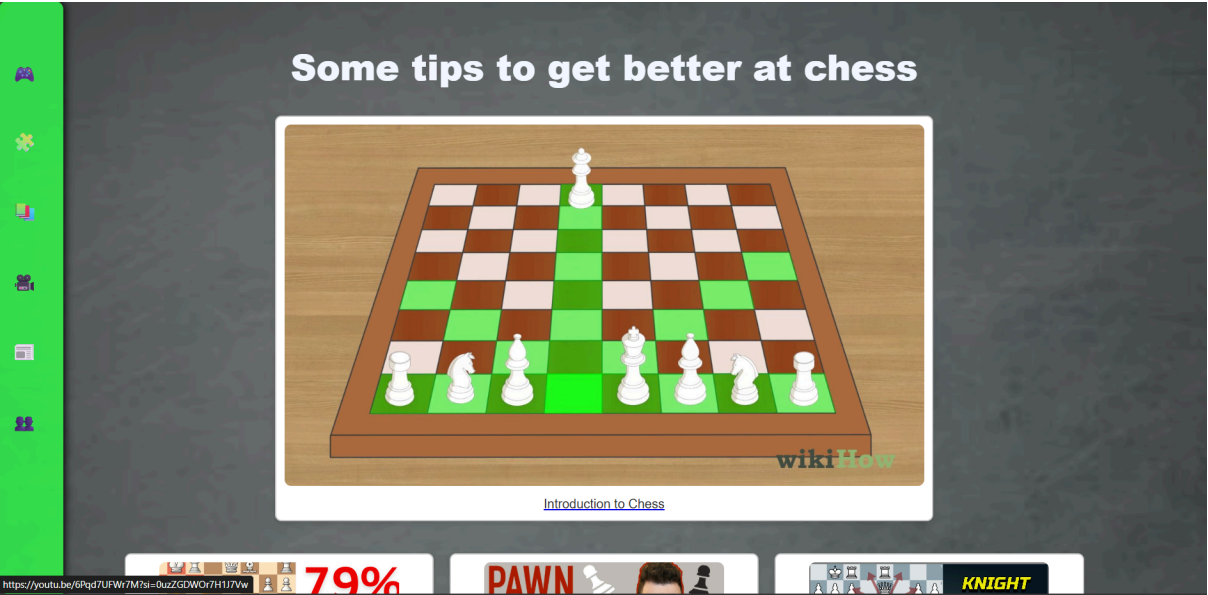
Ngoài những chế độ chơi để người chơi thử sức và giải trí web còn có những mục khác nhằm bổ sung và tăng thêm mức độ trải nghiệm cho người dùng.

Tất cả những web phụ đều tuân theo quy trình và xử lý giống với HTML gốc (index.html) ở phần sidebar và nền, các phần nội dung được hiển thị trong các khối trắng (**whitesmoke**) có viền và đổ bóng.

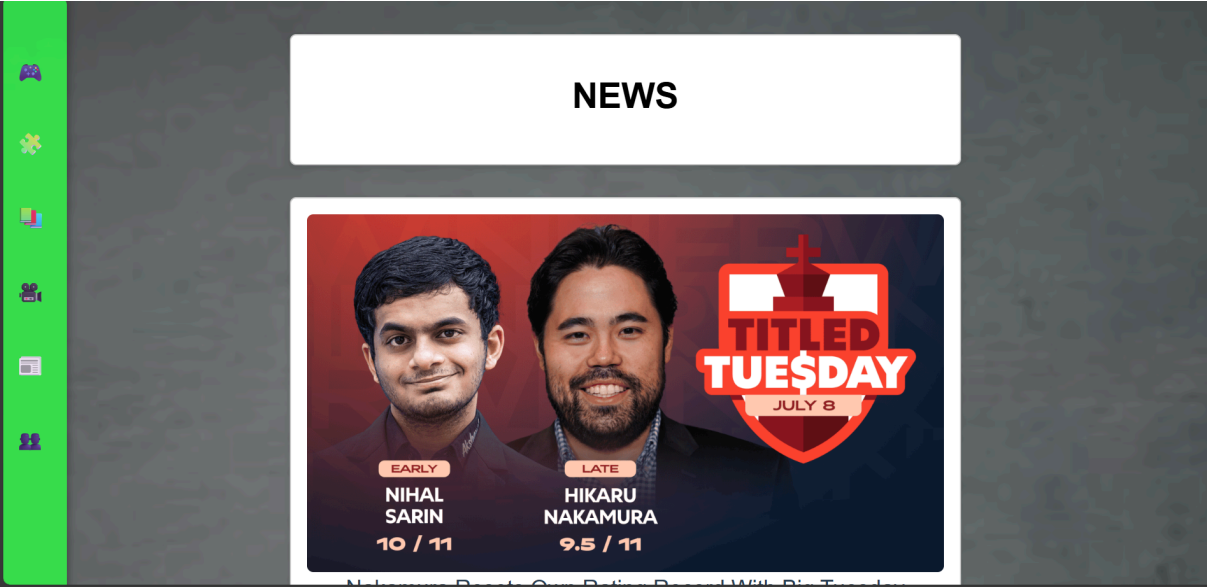
1/Learn



2/Watch



3/News



4/Social

Facebook • YouTube • Instagram

Follow the 5-time World Champion and chess icon Magnus Carlsen across platforms.



GothamChess (Levy Rozman)
Instagram

Popular chess streamer and educator. 2M+ followers.



Hikaru Nakamura

5-time U.S. Champion and top Twitch chess streamer.



Alireza Firouzja

Rising chess prodigy and World Championship contender.



R. Pragganandhaa
Inetanram



Andrea Botez



Alexandra Botez
Infatram

5/Puzzle